

A PROJECT REPORT ON

IOT-BASED SMART WATER QUALITY MONITORING AND ALERT SYSTEM

Domain: IoT / Environment

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 IoT Hardware Components	
	4.3 Software Specification	
V	SYSTEM ANALYSIS	11

	5.1 Feasibility Study	
	5.2 About the Software	
VI	SYSTEM DESIGN	16
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	25
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	33
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	34

CHAPTER I

INTRODUCTION

Safe drinking water is a basic human need, yet water-borne diseases such as diarrhoea, cholera, typhoid and hepatitis A continue to affect a large number of people in India every year. In villages, drinking water is supplied through overhead tanks, borewells and canals, and its quality can change suddenly because of sewage mixing, industrial discharge, pipeline leakage or poor tank maintenance.

At present, water quality is checked by collecting samples and sending them to a laboratory. This process is slow and costly, and is carried out only occasionally. Between two tests, contamination can go unnoticed for weeks and people continue to drink unsafe water.

The Internet of Things (IoT) makes it possible to measure physical and chemical properties of water continuously using low-cost sensors and to send the readings to the cloud through Wi-Fi or mobile networks. This project, AquaSense, uses pH, TDS, turbidity and temperature sensors connected to an ESP32 microcontroller to monitor water quality in real time, display the data on a dashboard and raise instant alerts through SMS and Telegram when water becomes unsafe.

LIMITATIONS

1. Laboratory testing is slow, costly and infrequent.
2. Village overhead tanks are rarely cleaned or tested regularly.
3. There is no live public data on drinking water quality.
4. Contamination is detected only after people fall sick.

1.1 History of Domain

Water quality monitoring started with manual chemical tests and microbiological culture in laboratories. In the 1970s and 1980s, electronic probes for pH, conductivity and dissolved oxygen became available and multi-parameter sondes were used by environmental agencies for river monitoring, but these instruments were expensive.

With the arrival of low-cost microcontrollers such as Arduino (2005) and ESP8266/ESP32 (2014-2016) with built-in Wi-Fi, along with inexpensive analog sensors, researchers began building affordable IoT water quality monitoring systems. Cloud platforms and messaging protocols such as MQTT now make it possible to monitor hundreds of sites from a single dashboard.

1.2 Overview of the Project

AquaSense consists of a sensor node and a cloud application. The sensor node, built around an ESP32, reads pH, TDS, turbidity and temperature every 30 seconds, filters noise using a moving average, and publishes the readings to an MQTT broker. A Node.js service subscribes to the readings, stores them in MongoDB, and checks each reading against WHO and BIS IS 10500 drinking water limits.

In addition to fixed thresholds, an Isolation Forest anomaly detection model identifies sudden abnormal changes, such as a rapid rise in turbidity after a pipeline break, even before the absolute limit is crossed. When water is unsafe, alerts are sent to the panchayat operator and registered residents through SMS and Telegram. A React dashboard shows live gauges, history graphs and weekly quality reports.

1.3 Motivation

Most water quality problems in villages are discovered only after an outbreak of disease. If the quality could be watched continuously and cheaply, the operator could act immediately.

- **Motivation:** To provide continuous, low-cost, real-time monitoring of drinking water sources with instant warnings.
- **Outcome:** Early detection of contamination, quick corrective action such as tank cleaning or chlorination, and transparent public data on water safety.

1.4 Aim of the Project

The project is entitled as “**IOT-BASED SMART WATER QUALITY MONITORING AND ALERT SYSTEM**”. The aim of this project is to design and implement a low-cost IoT system that continuously measures key water quality parameters, stores and visualises the data in the cloud, and automatically alerts responsible persons when the water becomes unsafe for drinking.

1.5 Objectives

- To interface pH, TDS, turbidity and temperature sensors with the ESP32.
- To transmit readings to the cloud using the MQTT protocol.
- To compare readings with WHO/BIS limits and detect anomalies with Isolation Forest.
- To send instant alerts through SMS and Telegram.
- To provide live dashboards and weekly reports for each site.

1.6 Literature Review

Vijayakumar and Ramya (2015): Proposed a real-time water quality monitoring system using sensors for pH, turbidity, conductivity and dissolved oxygen interfaced with a Raspberry Pi, and showed the data on the internet using IoT.

Pasika and Gandla (2020): Developed a smart water quality monitoring system with pH, turbidity, ultrasonic level and temperature sensors connected to a microcontroller with readings uploaded to the ThingSpeak cloud.

Liu, Ting and Zhou (2008): Introduced the Isolation Forest algorithm, which isolates anomalies by random partitioning rather than modelling normal points; anomalies require fewer partitions and therefore have shorter path lengths.

BIS IS 10500:2012: The Bureau of Indian Standards drinking water specification gives acceptable limits such as pH 6.5 to 8.5, TDS 500 mg/l and turbidity 1 NTU (permissible 5 NTU), which are used as the thresholds in this project.

WHO Guidelines for Drinking-water Quality (2017): The World Health Organization guidelines describe the health basis for water quality parameters and recommend continuous operational monitoring of water supply systems.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

In the existing system, water samples are collected manually by health workers or water board staff and taken to a laboratory for testing. Results take several days. Testing is done monthly or less often, and the results are stored in registers that are not visible to the public.

DISADVANTAGES

- Slow and infrequent testing.
- High cost per test and need for trained staff.
- No real-time warning; contamination is detected late.
- No historical data analysis.
- No public transparency.

2.2 Proposed System

The proposed system continuously monitors water quality using IoT sensors and an ESP32. Data is sent every 30 seconds to the cloud via MQTT and stored in MongoDB. The system compares readings with standard limits and uses an Isolation Forest model to detect abnormal changes. Alerts are sent immediately through SMS and Telegram and a dashboard shows live and historical data.

ADVANTAGES

- Continuous 24x7 monitoring at low cost (under Rs. 3,000 per node).
- Instant alerts when water becomes unsafe.
- Anomaly detection catches sudden changes early.
- Historical graphs help identify recurring problems.
- Scalable to many tanks, borewells and canals.

2.3 Project Schedule

The development of AquaSense follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the AquaSense system are listed below.

- **Sensor data acquisition:** Read pH, TDS, turbidity and temperature at a configurable interval.
- **Data transmission:** Publish readings to the MQTT broker with device ID and timestamp.
- **Data storage:** Store all readings in the database.
- **Threshold checking:** Compare each reading with WHO/BIS safe limits.
- **Anomaly detection:** Run Isolation Forest to detect sudden abnormal changes.
- **Alerting:** Send SMS/Telegram alerts with site name and parameter values.
- **Dashboard:** Show live values, gauges and history graphs.
- **Reports:** Generate weekly quality reports per site.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Reliability:** Readings buffered on the device if the network is lost and sent later.

- **Latency:** Alert must be delivered within 1 minute of an unsafe reading.
- **Low power:** Node should operate on a 12 V adapter or solar battery.
- **Accuracy:** Sensors calibrated with standard buffer solutions.
- **Security:** MQTT authentication with username and password; dashboard login.
- **Scalability:** Support at least 100 sensor nodes.

3.3 User Requirements

- View current water status of each site at a glance
- Receive alerts on mobile phone
- View history graphs for any date range
- Download weekly reports
- Add and manage sensor nodes

3.4 System Requirements

- ESP32 sensor node with Wi-Fi connectivity
- MQTT broker (Mosquitto)
- Node.js back-end service
- MongoDB database
- Web dashboard accessible via browser
- SMS gateway and Telegram bot

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System	: Intel i3 Processor 2.4 GHz or above
Hard Disk	: 256 GB
RAM	: 8 GB
Monitor	: 15" colour monitor
Microcontroller	: ESP32 DevKit V1
Power	: 12 V adapter / solar panel with battery

4.2 IoT Hardware Components

S.No	Component	Purpose
1	ESP32 DevKit V1	Main controller with Wi-Fi
2	Analog pH sensor (SEN0161)	Measures pH of water (0-14)
3	TDS sensor (Gravity)	Measures total dissolved solids in ppm
4	Turbidity sensor (SEN0189)	Measures cloudiness of water in NTU
5	DS18B20 waterproof probe	Measures water temperature
6	16x2 LCD with I2C	Shows local readings at the site
7	Buck converter (LM2596)	Converts 12 V to 5 V for the circuit

4.3 Software Specification

Operating System	: Windows 10 / Ubuntu 22.04
Firmware	: Arduino IDE (C/C++) for ESP32
Protocol	: MQTT (Mosquitto broker)
Back End	: Node.js with Express
ML Service	: Python 3.10, scikit-learn
Database	: MongoDB
Front End	: React JS with Chart.js / Grafana

Alerts : Twilio SMS, Telegram Bot API

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed AquaSense system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

The ESP32 and sensors are widely available and well documented, with Arduino libraries for each sensor. MQTT, Node.js and MongoDB are mature open-source technologies. The anomaly detection model is lightweight and runs on any server. Hence the system is technically feasible.

ECONOMIC FEASIBILITY

One sensor node costs less than Rs. 3,000, compared with several thousand rupees for repeated laboratory tests. All software is open source and can be hosted on a free-tier cloud server. The system is therefore economically feasible for panchayats and schools.

OPERATIONAL FEASIBILITY

Operators only need to look at the dashboard or read an alert message. Alerts are in simple language. Maintenance involves periodic sensor cleaning and calibration, which can be taught easily. Hence the system is operationally feasible.

5.2 About the Software

ESP32 MICROCONTROLLER

The ESP32 is a low-cost, low-power system-on-chip microcontroller from Espressif Systems with integrated Wi-Fi and dual-mode Bluetooth. It has a dual-core Tensilica Xtensa LX6 processor running up to 240 MHz, 520 KB SRAM, and many GPIO, ADC, I2C, SPI and UART pins.

The ESP32 can be programmed using the Arduino IDE in C/C++ or with MicroPython. Its built-in Wi-Fi lets sensor readings be sent directly to a cloud server using MQTT or HTTP without any additional module.

- Built-in Wi-Fi and Bluetooth
- 12-bit ADC for analog sensors
- Deep-sleep mode for battery operation
- Costs less than Rs. 500

WATER QUALITY SENSORS

The pH sensor (analog pH probe with signal board) measures the acidity or alkalinity of water on a scale of 0 to 14. The TDS sensor measures Total Dissolved Solids in parts per million (ppm) by measuring electrical conductivity. The turbidity sensor measures the cloudiness of water by detecting light scattered by suspended particles and outputs a value in NTU. The DS18B20 is a waterproof digital temperature sensor using the 1-Wire protocol.

All analog sensors are connected to the ESP32 ADC pins and calibrated with standard buffer solutions before deployment.

MQTT PROTOCOL

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol designed for constrained devices and low-bandwidth, high-latency or unreliable networks. It is an OASIS standard and is widely used in IoT.

Devices publish messages to topics on a broker (such as Mosquitto or HiveMQ), and any application subscribed to that topic receives the message instantly. MQTT supports three Quality of Service levels and retained messages.

- Very small packet overhead
- Publish-subscribe decoupling
- Works well on poor mobile networks

NODE.JS AND EXPRESS

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 engine. It uses an event-driven, non-blocking I/O model that makes it lightweight and efficient for data-intensive real-time applications.

Express.js is a minimal and flexible Node.js web framework that provides a robust set of features for building REST APIs, including routing, middleware and request/response handling. Together they allow both front end and back end to be written in JavaScript.

- Handles many concurrent connections
- npm - the largest package registry
- Ideal for real-time and IoT back ends

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI), which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

SCIKIT-LEARN

Scikit-learn is a free machine learning library for Python built on NumPy, SciPy and Matplotlib. It provides simple and efficient tools for predictive data analysis including classification, regression, clustering and dimensionality reduction.

It contains implementations of popular algorithms such as Random Forest, Support Vector Machine, Logistic Regression, k-Nearest Neighbours and Isolation Forest, along with utilities for train-test splitting, cross-validation, feature scaling, pipelines and evaluation metrics such as accuracy, precision, recall and F1-score.

- Consistent fit / predict API for all models
- Built-in preprocessing and model selection tools
- Well documented and production tested

MONGODB

MongoDB is a source-available, cross-platform, document-oriented NoSQL database. It stores data as flexible JSON-like documents (BSON), which means fields can vary from document to document and the data structure can be changed over time.

MongoDB supports indexing, aggregation pipelines, geospatial queries and time-series collections, which makes it well suited for storing sensor readings, user profiles and application logs.

- Schema-less flexible documents
- Horizontal scaling through sharding
- Powerful aggregation framework

REACT JS

React is an open-source JavaScript library for building user interfaces, maintained by Meta and a community of developers. It lets developers build reusable UI components and manages the view layer of web applications efficiently using a virtual DOM.

React applications are composed of components that hold their own state and re-render automatically when data changes. With libraries such as React Router and Axios, a single-page application can communicate with REST APIs and give the user a fast, app-like experience.

- Component based, reusable UI
- Virtual DOM for fast rendering
- Large ecosystem of UI libraries

GRAFANA

Grafana is an open-source analytics and interactive visualisation web application. It provides charts, graphs and alerts when connected to supported data sources such as InfluxDB, PostgreSQL and Prometheus.

Dashboards can be built without programming and alert rules can notify users through e-mail, Telegram or webhooks.

TWILIO SMS / TELEGRAM BOT API

Twilio is a cloud communications platform that offers APIs for sending SMS, making voice calls and sending WhatsApp messages from code. The Telegram Bot API is a free HTTP-based interface for sending messages to Telegram users and groups.

These services let the system deliver alerts directly to users' phones within seconds, even if they are not using the application at that time.

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of AquaSense is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

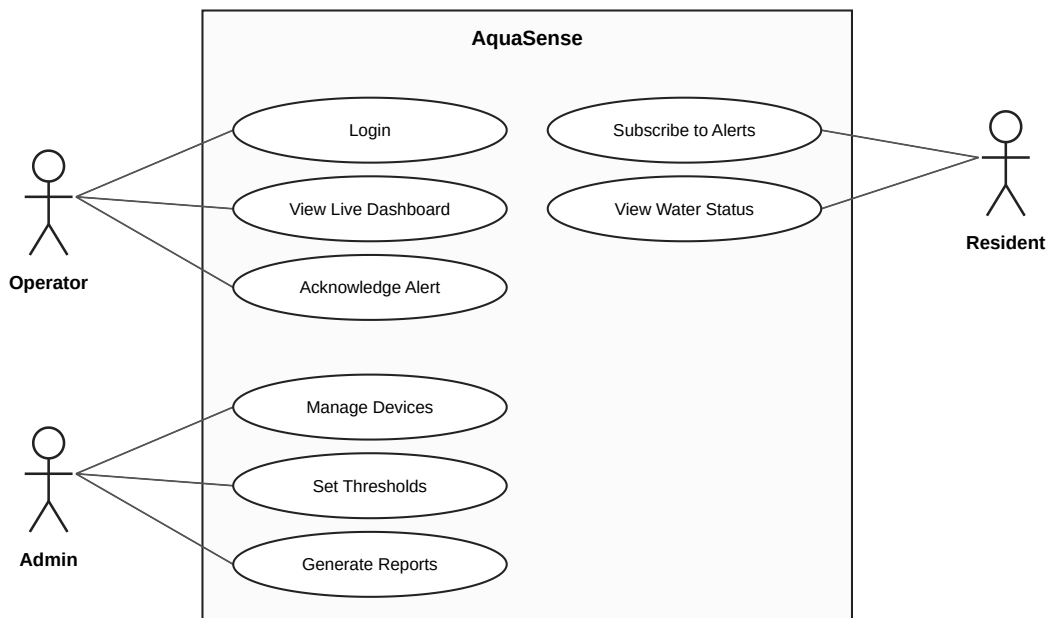


Fig 1: Use case diagram of AquaSense

6.2 System Architecture

The system has four layers. The sensing layer consists of sensors connected to the ESP32. The network layer uses Wi-Fi and the MQTT protocol to send data to the broker. The processing layer contains the Node.js service and the Python anomaly detection service. The application layer provides the dashboard and alert services.

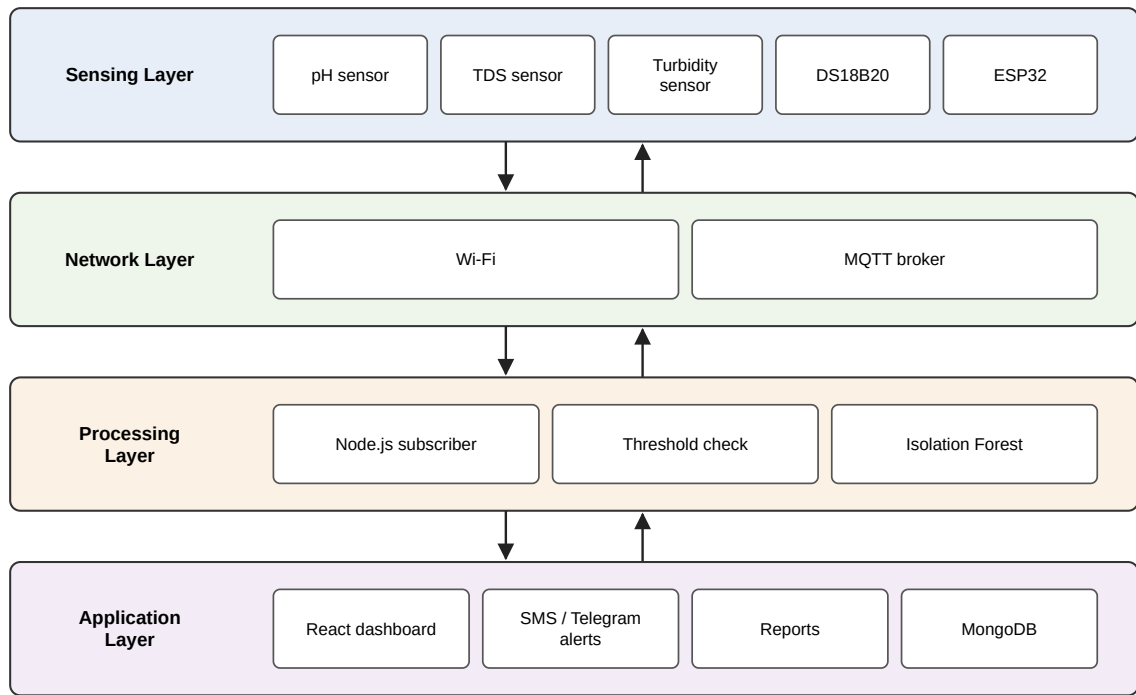


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

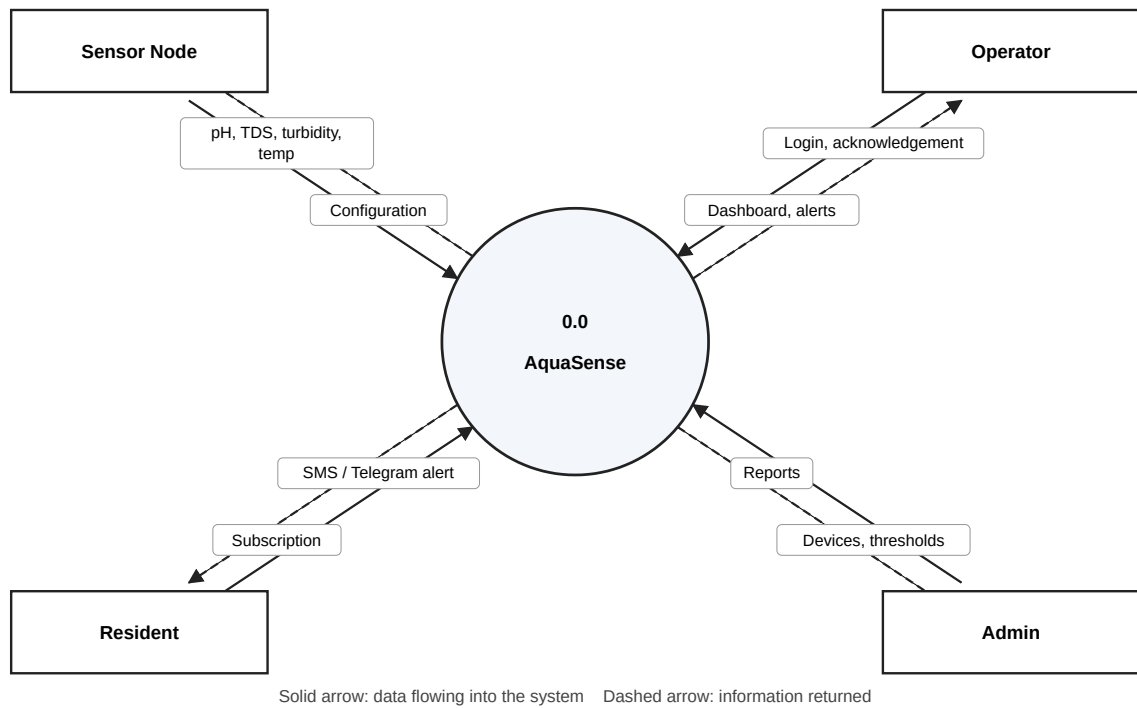


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

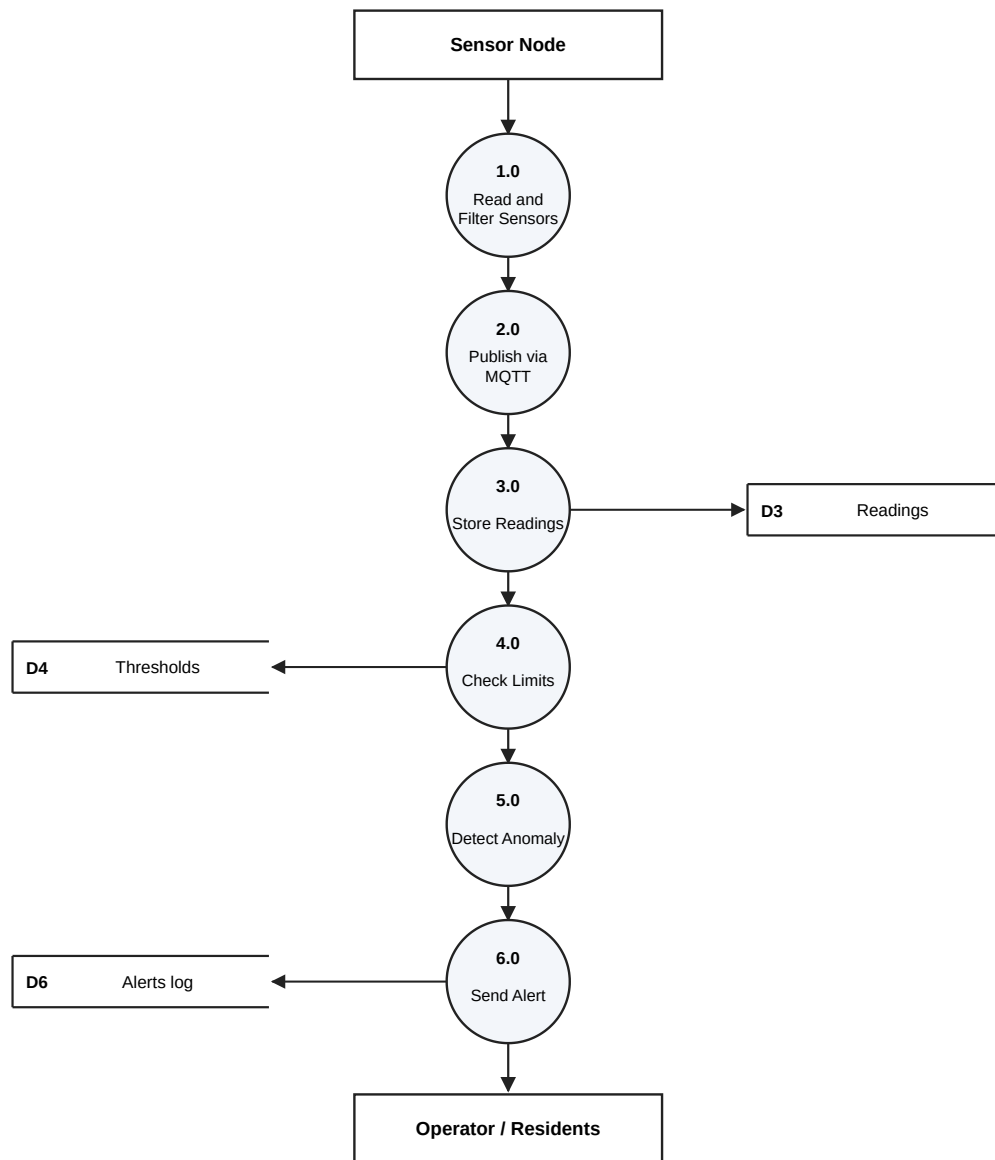


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

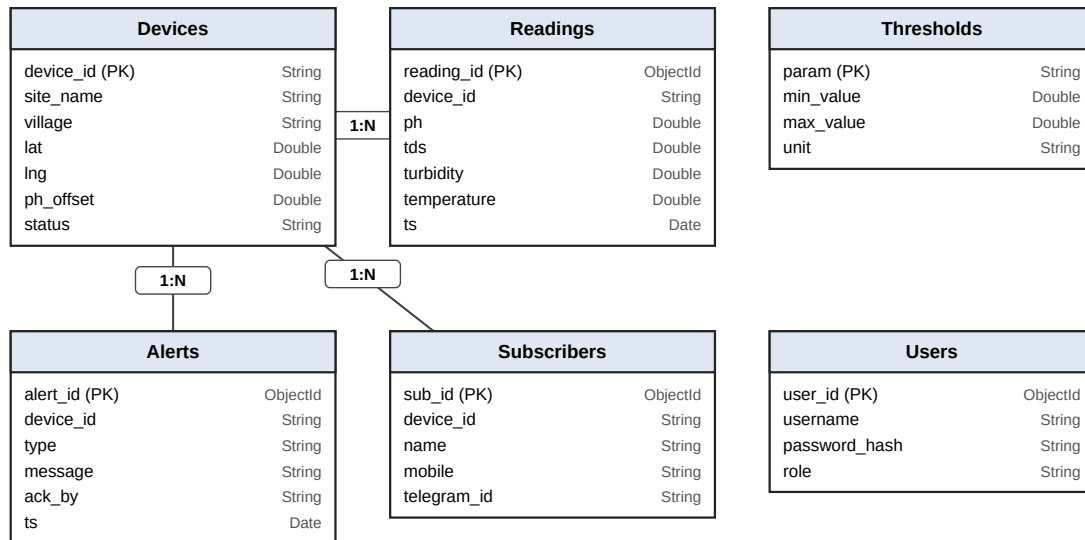


Fig 5: Entity relationship diagram

6.5 Table Design

DEVICES

FIELD NAME	DATA TYPE	DESCRIPTION
device_id	String	Primary key (e.g. AQ-001)
site_name	String	Tank / borewell name
village	String	Village
lat	Double	Latitude
lng	Double	Longitude
ph_offset	Double	Calibration offset
status	String	active / inactive

READINGS

FIELD NAME	DATA TYPE	DESCRIPTION
reading_id	ObjectId	Primary key
device_id	String	Foreign key to Devices
ph	Double	pH value
tds	Double	TDS in ppm
turbidity	Double	Turbidity in NTU

temperature	Double	Temperature in C
ts	Date	Timestamp

THRESHOLDS

FIELD NAME	DATA TYPE	DESCRIPTION
param	String	Primary key - parameter name
min_value	Double	Minimum safe value
max_value	Double	Maximum safe value
unit	String	Unit

ALERTS

FIELD NAME	DATA TYPE	DESCRIPTION
alert_id	ObjectId	Primary key
device_id	String	Foreign key to Devices
type	String	threshold / anomaly
message	String	Alert text
ack_by	String	Operator who acknowledged
ts	Date	Timestamp

SUBSCRIBERS

FIELD NAME	DATA TYPE	DESCRIPTION
sub_id	ObjectId	Primary key
device_id	String	Foreign key to Devices
name	String	Resident name
mobile	String	Mobile number
telegram_id	String	Telegram chat ID

USERS

FIELD NAME	DATA TYPE	DESCRIPTION
user_id	ObjectId	Primary key

username	String	Login name
password_hash	String	Hashed password
role	String	operator / admin

6.6 Algorithm Design

The monitoring algorithm runs partly on the ESP32 (sensing and filtering) and partly on the server (checking, anomaly detection and alerting).

1. **Read sensors every N seconds** – The ESP32 reads analog values from the pH, TDS and turbidity sensors and the digital DS18B20 every 30 seconds.
2. **Calibrate and filter noise (moving average)** – Ten samples are averaged and calibration equations convert voltage to pH, ppm and NTU; TDS is temperature compensated.
3. **Send data to cloud via MQTT/HTTP** – A JSON message is published to topic `aquasense/<device_id>/data`.
4. **Compare with WHO/BIS safe limits** – Each parameter is compared with the configured minimum and maximum values.
5. **Run Isolation Forest to detect sudden abnormal changes** – The last readings and their rate of change are scored; a score below the cut-off indicates an anomaly.
6. **Trigger alert if unsafe** – An SMS and Telegram message is sent to the operator and subscribers, with a 30-minute cool-down to avoid repeated messages.
7. **Store history and show live graphs** – Readings are stored in MongoDB and pushed to the dashboard via WebSocket.
8. **Generate weekly quality report** – Weekly averages, maxima and unsafe events are computed with aggregation queries.

PSEUDO CODE

```

LOOP every 30 seconds (ESP32)
  FOR each sensor: v <- AVERAGE(10 analog samples)
  ph <- 7 + (2.5 - v_ph) / 0.18 + offset
  tds <- TDS_FORMULA(v_tds, temperature)
  ntu <- TURBIDITY_CURVE(v_turb)

```

```

    PUBLISH("aquasense/id/data", {ph, tds, ntu, temperature})
END LOOP

ON MESSAGE (server)
    SAVE reading
    unsafe <- any parameter outside [min, max]
    anomaly <- ISOLATION_FOREST.score(features) < cutoff
    IF (unsafe OR anomaly) AND no alert in last 30 min THEN SEND_ALERT
    EMIT reading to dashboard

```

6.7 Modules

The system is divided into the hardware module and the software modules for the Operator, Resident and Admin users.

MODULES

OPERATOR

1. LOGIN
2. VIEW LIVE DASHBOARD
3. ACKNOWLEDGE ALERT

RESIDENT

1. SUBSCRIBE TO ALERTS
2. VIEW WATER STATUS

ADMIN

1. MANAGE DEVICES
2. SET THRESHOLDS
3. GENERATE REPORTS

OPERATOR

LOGIN

The panchayat water operator logs in with username and password to access the dashboard of the sites assigned to them.

VIEW LIVE DASHBOARD

Displays the latest pH, TDS, turbidity and temperature of each site with colour-coded status: green for safe, orange for warning and red for unsafe.

ACKNOWLEDGE ALERT

When an alert is raised, the operator acknowledges it and records the action taken, such as tank cleaning or chlorination.

RESIDENT

SUBSCRIBE TO ALERTS

Residents register their mobile number or Telegram ID for a site to receive alerts when water becomes unsafe.

VIEW WATER STATUS

A public status page shows whether the water at a site is currently safe.

ADMIN

MANAGE DEVICES

Admin registers new sensor nodes with site name, location and calibration constants.

SET THRESHOLDS

Admin configures safe limits for each parameter as per BIS/WHO standards.

GENERATE REPORTS

Admin generates weekly and monthly reports with averages, maximum values and the number of unsafe events.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

The AquaSense system was implemented in two parts: firmware for the ESP32 written in C++ using the Arduino IDE, and a cloud application consisting of a Node.js MQTT subscriber, a Python anomaly detection microservice, MongoDB and a React dashboard. The sensors were calibrated using pH 4.0, 7.0 and 9.18 buffer solutions and a TDS reference solution of 342 ppm.

7.2 Implementation

The Isolation Forest model was trained on two weeks of normal readings collected from a school overhead tank. Features included current values and the difference from the previous reading. Testing was done by deliberately adding salt, soil and lemon juice to sample water to simulate contamination.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	pH reading accuracy	pH 7.0 buffer solution	Reading 6.9 - 7.1	Pass
TC02	TDS reading	342 ppm reference solution	Reading within +/-5%	Pass
TC03	MQTT publish	Node powered on	Reading appears in dashboard within 5 s	Pass
TC04	Unsafe pH alert	Lemon juice added (pH 5.2)	SMS and Telegram alert received	Pass
TC05	Turbidity anomaly	Soil added suddenly	Anomaly alert before 5 NTU limit	Pass
TC06	Network loss	Wi-Fi switched off for 5 min	Buffered readings sent after reconnect	Pass
TC07	Duplicate alert control	Unsafe for 20 min	Only one alert in 30 min window	Pass
TC08	Weekly report	Select site and week	PDF report with averages generated	Pass

7.4 Source Code

aquasense_node.ino (ESP32 firmware)

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <OneWire.h>
#include <DallasTemperature.h>

#define PH_PIN 34
#define TDS_PIN 35
#define TURB_PIN 32
#define TEMP_PIN 4
const char* DEVICE_ID = "AQ-001";

WiFiClient net;
PubSubClient mqtt(net);
OneWire oneWire(TEMP_PIN);
DallasTemperature tempSensor(&oneWire);

float readAvg(int pin) {
```

```

    long sum = 0;
    for (int i = 0; i < 10; i++) { sum += analogRead(pin); delay(20); }
    return (sum / 10.0) * 3.3 / 4095.0;          // voltage
}

void setup() {
    Serial.begin(115200);
    WiFi.begin("SSID", "PASSWORD");
    while (WiFi.status() != WL_CONNECTED) delay(500);
    mqtt.setServer("broker.aquasense.local", 1883);
    tempSensor.begin();
}

void loop() {
    if (!mqtt.connected()) mqtt.connect(DEVICE_ID, "aqnode", "secret");
    tempSensor.requestTemperatures();
    float t = tempSensor.getTempCByIndex(0);
    float ph = 7.0 + (2.5 - readAvg(PH_PIN)) / 0.18;
    float v = readAvg(TDS_PIN);
    float comp = v / (1.0 + 0.02 * (t - 25.0));
    float tds = (133.42 * comp * comp * comp - 255.86 * comp * comp + 857.39 *
comp) * 0.5;
    float vt = readAvg(TURB_PIN) * 1.5;          // scale to 0-4.5V sensor range
    float ntu = max(0.0, -1120.4 * vt * vt + 5742.3 * vt - 4352.9) / 100.0;
    char msg[160];
    snprintf(msg, sizeof msg, "
{"ph":%.2f,"tds":%.0f,"turbidity":%.2f,"temperature":%.1f}", ph, tds,
ntu, t);
    mqtt.publish("aquasense/AQ-001/data", msg);
    mqtt.loop();
    delay(30000);
}

```

server.js (Node.js subscriber and alert service)

```

const mqtt = require("mqtt");
const axios = require("axios");
const { MongoClient } = require("mongodb");
const twilio = require("twilio")(process.env.TW_SID, process.env.TW_TOKEN);

const LIMITS = { ph: [6.5, 8.5], tds: [0, 500], turbidity: [0, 5] };
const lastAlert = {};

(async () => {
    const db = (await
MongoClient.connect("mongodb://localhost:27017")).db("aquasense");
    const client = mqtt.connect("mqtt://localhost", { username: "server",
password: "secret" });
    client.on("connect", () => client.subscribe("aquasense+/data"));

    client.on("message", async (topic, payload) => {
        const deviceId = topic.split("/")[1];
        const r = { deviceId, ...JSON.parse(payload.toString()), ts: new Date() };
        await db.collection("readings").insertOne(r);

        const problems = Object.entries(LIMITS)
            .filter(([k, [lo, hi]]) => r[k] < lo || r[k] > hi)
            .map(([k]) => `${k}=${r[k]}`);
        const { data } = await axios.post("http://localhost:8000/score", r);    //
Python Isolation Forest
        if (data.anomaly) problems.push("sudden abnormal change");
    });
})();

```

```

    const now = Date.now();
    if (problems.length && (!lastAlert[deviceId] || now - lastAlert[deviceId] >
30 * 60000)) {
        lastAlert[deviceId] = now;
        const dev = await db.collection("devices").findOne({ device_id: deviceId
});
        const text = `ALERT: Water at ${dev.site_name} is UNSAFE
(${problems.join(", ")}). Do not drink.`;
        const subs = await db.collection("subscribers").find({ device_id: deviceId
}).toArray();
        for (const s of subs) {
            if (s.mobile) await twilio.messages.create({ to: s.mobile, from:
process.env.TW_FROM, body: text });
            if (s.telegram_id) await
axios.post(`https://api.telegram.org/bot${process.env.TG}/sendMessage`,
                { chat_id: s.telegram_id, text });
        }
        await db.collection("alerts").insertOne({ device_id: deviceId, message:
text, ts: new Date() });
    }
  });
})();

```

anomaly_service.py

```

from fastapi import FastAPI
from pydantic import BaseModel
import joblib, numpy as np

app = FastAPI()
model = joblib.load("iforest.pkl")          # trained on normal readings
prev = {}

class Reading(BaseModel):
    deviceId: str
    ph: float
    tds: float
    turbidity: float
    temperature: float

@app.post("/score")
def score(r: Reading):
    cur = np.array([r.ph, r.tds, r.turbidity, r.temperature])
    delta = cur - prev.get(r.deviceId, cur)
    prev[r.deviceId] = cur
    s = model.decision_function([np.concatenate([cur, delta])])[0]
    return {"score": float(s), "anomaly": bool(s < -0.05)}

```

7.5 Screen Layout

SCREENSHOTS:

LOGIN

https://aquasense.app/

AquaSense

[Dashboard](#)[Live Data](#)[Alerts](#)[Reports](#)[Devices](#)

AquaSense Operator Login

Username

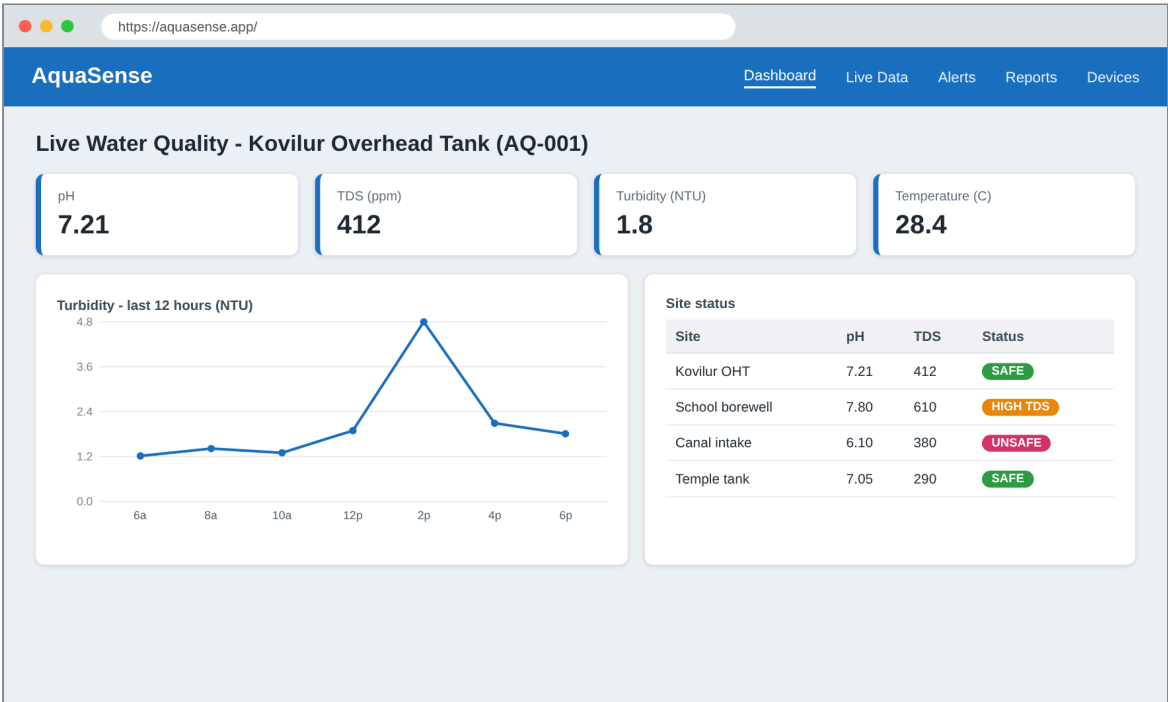
operator.kovilur

Password

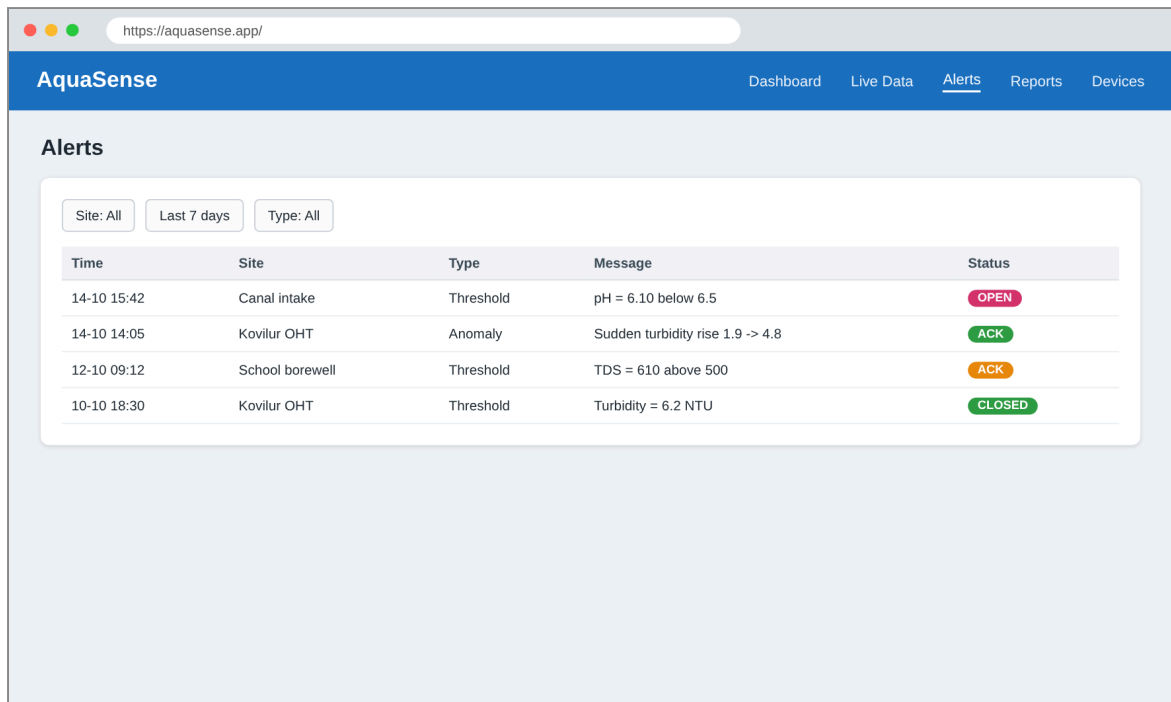
Login

[Forgot password?](#)

LIVE DASHBOARD



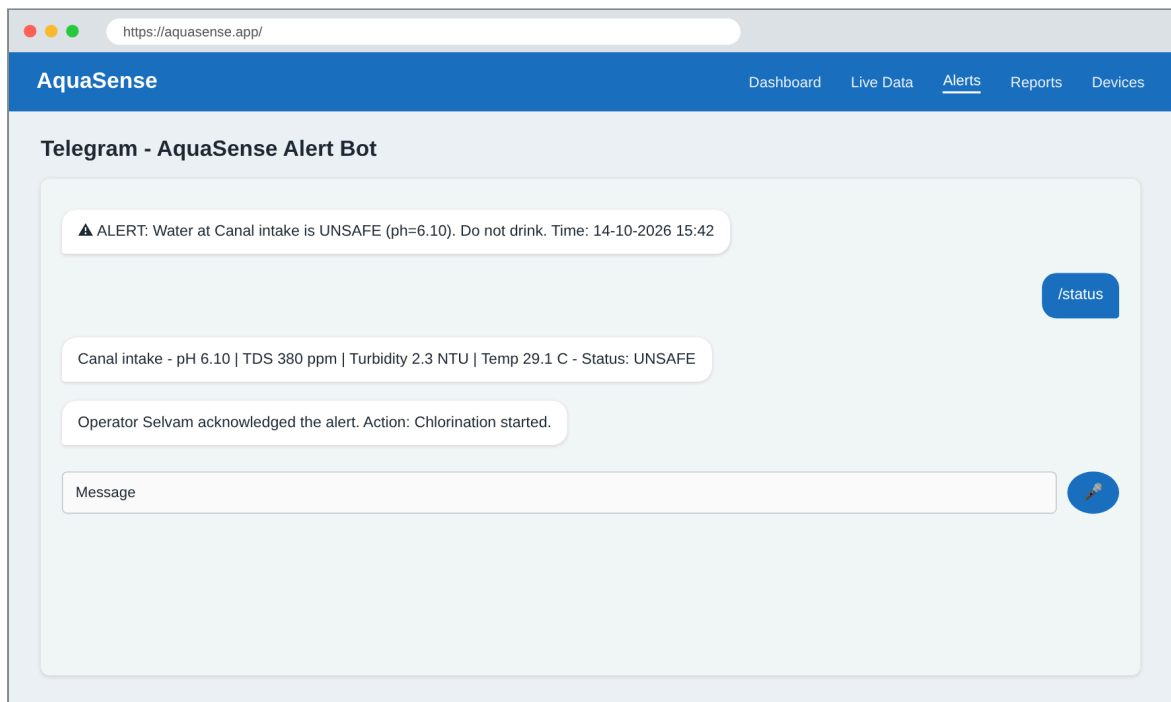
ALERT HISTORY



The screenshot shows the AquaSense web application interface. The top navigation bar includes 'Dashboard', 'Live Data', 'Alerts' (selected), 'Reports', and 'Devices'. The main section is titled 'Alerts' and contains a filter bar with 'Site: All', 'Last 7 days', and 'Type: All'. Below the filters is a table with the following data:

Time	Site	Type	Message	Status
14-10 15:42	Canal intake	Threshold	pH = 6.10 below 6.5	OPEN
14-10 14:05	Kovilur OHT	Anomaly	Sudden turbidity rise 1.9 -> 4.8	ACK
12-10 09:12	School borewell	Threshold	TDS = 610 above 500	ACK
10-10 18:30	Kovilur OHT	Threshold	Turbidity = 6.2 NTU	CLOSED

TELEGRAM ALERT

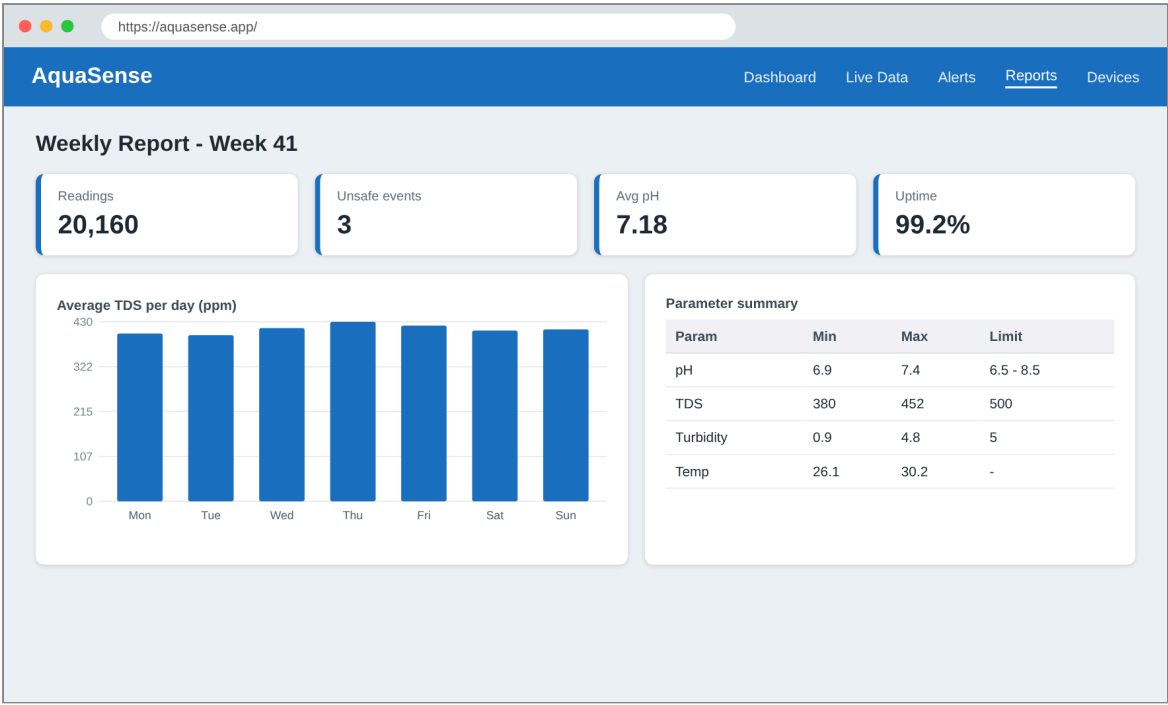


The screenshot shows the Telegram - AquaSense Alert Bot interface. The top navigation bar is the same as the previous screenshot. The main section is titled 'Telegram - AquaSense Alert Bot' and contains a chat log with the following messages:

- ▲ ALERT: Water at Canal intake is UNSAFE (ph=6.10). Do not drink. Time: 14-10-2026 15:42
- Canal intake - pH 6.10 | TDS 380 ppm | Turbidity 2.3 NTU | Temp 29.1 C - Status: UNSAFE
- Operator Selvam acknowledged the alert. Action: Chlorination started.

At the bottom, there is a 'Message' input field and a blue circular button with a microphone icon. A '/status' button is also visible on the right side of the chat log.

WEEKLY REPORT



CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The IoT-based Smart Water Quality Monitoring and Alert System continuously measures pH, TDS, turbidity and temperature using low-cost sensors and an ESP32, transmits the data through MQTT, and alerts operators and residents within a minute of the water becoming unsafe. The Isolation Forest model detected sudden contamination events before the absolute limits were crossed.

The system is inexpensive, easy to install and scalable to many sites. It provides transparent, continuous information on drinking water quality and can help prevent water-borne disease outbreaks in villages and schools.

8.2 Future Enhancement

- Add dissolved oxygen, residual chlorine and nitrate sensors.
- Use LoRaWAN or GSM for sites without Wi-Fi.
- Solar-powered nodes with deep-sleep for remote canals.
- Automatic chlorine dosing pump controlled by the system.
- Public mobile app showing water quality of all village sources.

CHAPTER IX

BIBLIOGRAPHY

1. Vijayakumar, N., and Ramya, R., "The real time monitoring of water quality in IoT environment", Proc. IEEE ICIIECS, 2015.
2. Pasika, S., and Gandla, S. T., "Smart water quality monitoring system with cost-effective using IoT", Heliyon, Vol. 6, No. 7, e04096, 2020.
3. Liu, F. T., Ting, K. M., and Zhou, Z.-H., "Isolation Forest", Proc. IEEE International Conference on Data Mining (ICDM), pp. 413-422, 2008.
4. Bureau of Indian Standards, "IS 10500:2012 Drinking Water - Specification (Second Revision)", New Delhi, 2012.
5. World Health Organization, Guidelines for Drinking-water Quality, 4th Edition incorporating the 1st addendum, Geneva, 2017.
6. Banks, A., and Gupta, R. (Eds.), MQTT Version 3.1.1, OASIS Standard, 2014.

WEBSITE REFERENCE

1. <https://docs.espressif.com>
2. <https://mosquitto.org>
3. <https://www.mongodb.com/docs>
4. <https://scikit-learn.org>